DX Front-End Template

Volume 2 - Front-End Template Guide

React 19 | TypeScript | Vite 8 | Tailwind 4 | TanStack Query | Zustand

Bagian dari DNIA DX Development Starter Kit. Panduan ini membantu intern memahami arsitektur front-end berbasis Feature MVC, cara menjalankan lingkungan lokal, pola konsumsi API, kontrol keamanan, serta quality gate sebelum membuat perubahan.

> **Pesan utama**
> Gunakan template ini sebagai baseline UI. Bangun fitur baru dengan pola model, controller, dan view. Pertahankan kontrol keamanan komunikasi, validasi, dan quality gate; ganti hanya domain bisnisnya.

Audience: DX Intern / Junior Front-End Developer
Document owner: PED / Digital Transformation
Classification: Internal / Proprietary

1. Apa Itu DX Front-End Template?

DX Front-End Template adalah fondasi antarmuka internal untuk React 19, TypeScript, Vite 8, dan Tailwind CSS 4. Aplikasi contoh terhubung ke API Node MVC untuk autentikasi dan Work Order.

Tujuan	Makna untuk Intern
Mempercepat pembuatan UI	Routing, layout, notifikasi, data fetching, dan styling sudah tersedia.
Menstandarkan arsitektur	Setiap fitur mengikuti pola model, controller, dan view yang seragam.
Komunikasi aman	Axios withCredentials, signed CSRF, JWT hanya pada HttpOnly cookie, dan tanpa dangerouslySetInnerHTML.
Menjaga kualitas	ESLint, Vitest, build check, npm audit, dan SonarQube menjadi kontrol sebelum handover.

Batas penggunaan

- Template berstatus private dan UNLICENSED.
- Source code, konfigurasi, credential, dan dokumentasi tidak boleh didistribusikan tanpa otorisasi tertulis.
- Backend tetap menjadi security boundary; validasi front-end bukan pengganti validasi server.

2. Teknologi Utama

Area	Teknologi / Paket	Kegunaan
Runtime	Node.js >=22.12.0 <23	Runtime yang diwajibkan oleh project.
UI framework	React 19	Membangun komponen antarmuka.
Language	TypeScript 5	Type safety di seluruh aplikasi.
Build tool	Vite 8	Dev server cepat dan build produksi.
Styling	Tailwind CSS 4	Utility-first styling.
Server state	TanStack Query 5	Fetching, caching, dan sinkronisasi data server.
Client state	Zustand 5	State aplikasi ringan di sisi client.
Form + validation	React Hook Form + Zod	Form handling dan strict schema.

Area	Teknologi / Paket	Kegunaan
HTTP client	Axios	Komunikasi REST dengan withCredentials.
Realtime	Socket.IO Client	Invalidasi cache dari SelfiNotify.
Notification	Sonner	Toast dan feedback UI.
Icons	Lucide React	Ikon konsisten.
Testing	Vitest + Testing Library	Unit dan component test.
Code quality	ESLint + SonarQube	Linting dan static analysis.

3. Gambaran Arsitektur

```
User
  -> View (page / component)
  -> Controller (hooks / orchestration)
  -> Model (API / schema / type / state)
  -> Axios (withCredentials + CSRF)
  -> Node MVC Backend
  -> SQL Server
```

Aturan inti
View menampilkan data dan menangani interaksi. Controller mengatur data fetching dan business orchestration melalui hooks. Model menyimpan API call, schema, type, dan state. Komponen tidak memanggil API mentah langsung dari view.

4. Struktur Folder dan Tanggung Jawab

Folder / File	Tanggung Jawab
src/app	Wiring aplikasi: App.tsx (providers) dan router.tsx (routing).
src/config	Konfigurasi runtime, misalnya env.ts untuk environment variable.
src/features	Modul fitur berbasis MVC (auth, work-orders).
src/features/<fitur>/model	API call, schema Zod, type, dan state fitur.
src/features/<fitur>/controller	Custom hooks dan orchestration business logic.
src/features/<fitur>/view	Page dan komponen UI fitur.

Folder / File	Tanggung Jawab
src/shared/components/layout	Layout lintas fitur, misalnya AppLayout.
src/shared/components/ui	Komponen UI reusable.
src/shared/lib	api.ts, csrf.ts, queryClient.ts, dan utils.ts.
src/styles	Styling global dan konfigurasi Tailwind.
src/test	Automated test.
src/main.tsx	Entry point aplikasi React.
scripts	Runtime check dan automation.

5. Application Bootstrap dan Routing

App.tsx mendaftarkan QueryClientProvider, RouterProvider, dan Toaster. router.tsx mendefinisikan route publik dan protected.

```
createBrowserRouter([
  { path: "/login", element: <LoginPage /> },
  { element: <ProtectedRoute />, children: [
    { element: <AppLayout />, children: [
      { index: true, element: <WorkOrdersPage /> }
    ]}
  ]}
])
```

Route	Akses	Tujuan
/login	Public	Halaman autentikasi.
/	Protected	Dibungkus ProtectedRoute dan AppLayout.
/ (index)	Protected	Halaman Work Orders.

6. Feature-Based MVC

Setiap fitur mandiri dan memiliki tiga layer. Pola ini sejajar dengan MVC pada backend sehingga intern mudah berpindah antara front-end dan back-end.

Layer	Isi	Contoh
model	API, schema, type, state	Definisi request Work Order dan schema Zod.

Layer	Isi	Contoh
controller	Hooks dan orchestration	useWorkOrders untuk fetch, create, update.
view	Page dan komponen	WorkOrdersPage, LoginPage, ProtectedRoute.

```
features/
  auth/
    model/  controller/  view/
  work-orders/
    model/  controller/  view/
```

7. State Management

Jenis State	Tools	Kegunaan
Server state	TanStack Query	Cache, refetch, loading, dan error state data server.
Client state	Zustand	State UI ringan, misalnya toggle dan preferensi.
Form state	React Hook Form + Zod	Input, validasi, dan error form.

Preferensi lokal
localStorage hanya dipakai untuk preferensi seperti dark mode, bukan untuk menyimpan data autentikasi.

8. Komunikasi API dan Keamanan

Kontrol	Penerapan
Axios	Selalu memakai withCredentials.
CSRF	Signed CSRF token ditambahkan interceptor pada request mutasi.
JWT	Hanya pada HttpOnly cookie; tidak ada JWT di localStorage.
XSS	Data server dirender sebagai plain text; tanpa dangerouslySetInnerHTML.
Validation	Zod memvalidasi form; backend tetap security

Kontrol	Penerapan
	boundary.
Realtime	SelfiNotify hanya meng-invalidasi cache TanStack Query; bila socket terputus, REST tetap berjalan dengan fallback indicator.

9. Fitur Contoh yang Tersedia

- Login dan protected route.
- Dashboard ringkas dengan realtime indicator.
- Table responsive dengan search, filter, dan pagination.
- Create/edit dan delete confirmation berbasis role.
- Export Excel dan PDF.
- Toast, dark mode, serta loading, empty, dan error state.

10. Quick Start Lokal

Ikuti urutan berikut. Jangan commit file .env atau credential lokal.

```
nvm use
npm ci
Copy-Item .env.example .env
npm run dev
```

Set VITE_API_BASE_URL ke endpoint API. Runtime yang didukung adalah Node 22.12+ dengan baseline verifikasi 22.23.1.

> **Catatan**
> Origin front-end wajib didaftarkan pada CORS backend agar komunikasi berhasil.

11. Quality Gate dan Testing

Perintah / Control	Apa yang Diverifikasi
npm run lint	ESLint dan aturan React Hooks.
npm test	Component dan unit test dengan Vitest.
npm run build	Type check (tsc -b) dan build Vite ke dist/.
npm run check	Menjalankan lint, test, dan build.

Perintah / Control	Apa yang Diverifikasi
npm audit	Memeriksa kerentanan dependency.
npm run sonar	Menjalankan SonarQube dari konfigurasi environment.

12. Build dan Deployment

- Build produksi menghasilkan folder dist/.
- Sajikan dist/ melalui web server dengan SPA fallback.
- Aktifkan HTTPS dan Content Security Policy yang sesuai.
- Gunakan cache immutable untuk asset ber-hash.
- Daftarkan origin front-end pada CORS backend.

13. Workflow Kontribusi Intern

Tahap	Yang Harus Dilakukan
1. Pahami tiket	Identifikasi fitur, model, controller, dan view yang terdampak.
2. Buat branch	Buat branch dari main.
3. Ikuti pola MVC	Letakkan API/schema di model, hooks di controller, UI di view.
4. Reuse shared	Gunakan komponen dan lib di shared, jangan menduplikasi.
5. Validasi input	Gunakan React Hook Form dan Zod pada form.
6. Tambahkan test	Tambahkan test untuk perubahan perilaku atau UI penting.
7. Jalankan quality gate	Jalankan npm run check sebelum pull request.
8. Buat pull request	Jelaskan perubahan, dampak UI/API, dan bukti test.

14. Yang Boleh dan Tidak Boleh

Do	Do Not
Gunakan Node 22 sesuai engine project.	Jangan memakai runtime lain tanpa persetujuan.
Simpan JWT hanya melalui HttpOnly cookie.	Jangan menyimpan JWT atau data auth di localStorage.

Do	Do Not
Render data server sebagai plain text.	Jangan memakai dangerouslySetInnerHTML.
Konsumsi API melalui shared/lib dan controller.	Jangan memanggil API mentah langsung dari komponen view.
Gunakan environment variable via config/env.ts.	Jangan hardcode endpoint atau secret di komponen.
Validasi form dengan Zod.	Jangan mengandalkan validasi front-end sebagai satu-satunya kontrol.

15. Tugas Onboarding yang Disarankan

Checklist berikut adalah rekomendasi latihan, bukan requirement eksplisit dari repository.

- [ ] Jalankan dev server dan buka halaman login.
- [ ] Login dan amati alur ProtectedRoute menuju Work Orders.
- [ ] Telusuri satu fitur dari view ke controller ke model.
- [ ] Tambahkan satu kolom atau filter pada table Work Order melalui model, controller, dan view.
- [ ] Tambahkan satu test dengan Testing Library.
- [ ] Jalankan npm run check dan perbaiki seluruh error.

16. Kapan Harus Bertanya kepada Mentor?

- Perubahan menyentuh authentication, CSRF, cookie, atau CORS.
- Perubahan membutuhkan endpoint atau kontrak API baru dari backend.
- Fitur memerlukan integrasi realtime atau data produksi.
- Requirement mendorong penyimpanan token di browser atau penggunaan dangerouslySetInnerHTML.
- npm audit atau SonarQube menemukan issue yang belum dipahami.

17. Referensi Repository

Dokumen ini disusun dari file template front-end berikut:

- README.md
- package.json
- src/app/App.tsx
- src/app/router.tsx
- src/config/env.ts
- src/features/auth (model, controller, view)
- src/features/work-orders (model, controller, view)
- src/shared/components/layout
- src/shared/components/ui
- src/shared/lib/api.ts
- src/shared/lib/csrf.ts

- src/shared/lib/queryClient.ts
- src/shared/lib/utils.ts

> **Sebelum mulai coding**
> Baca README.md, jalankan project secara lokal, dan pahami pola MVC per fitur. Jangan melewati quality gate hanya untuk mempercepat penyelesaian.